

F24-187-R-CodeCommClassifier

Project Team

Muhammad Riyaan Tariq	21I-1101
Abdur Rehman Afzal	21I-1151
Muhammad Haris Athar	21I-1159

Session 2021-2025

Supervised by

Mr. Pir Sami Ullah Shah



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2025

Contents

1	Introduction	1
1.1	Problem Domain	1
1.2	Research Problem Statement	2
2	Literature Review	3
2.1	Related Research	3
2.1.1	Performance Comparison of Binary Machine Learning Classifiers in Identifying Code Comment Types: An Exploratory Study [5] .	4
2.1.1.1	Summary of the Research Item	4
2.1.1.2	Critical Analysis of the Research Item	4
2.1.1.3	Relationship to the Proposed Research Work	5
2.1.2	Classifying Code Comments via Pre-trained Programming Lan- guage Model [8]	5
2.1.2.1	Summary of the Research Item	5
2.1.2.2	Critical Analysis of the Research Item	5
2.1.2.3	Relationship to the Proposed Research Work	6
2.1.3	STACC: Code Comment Classification using SentenceTransform- ers [1]	6
2.1.3.1	Summary of the Research Item	6
2.1.3.2	Critical Analysis of the Research Item	7
2.1.3.3	Relationship to the Proposed Research Work	7
2.1.4	Dopamin: Transformer-based Comment Classifiers through Do- main Post-Training and Multi-level Layer Aggregation [4]	8
2.1.4.1	Summary of the Research Item	8
2.1.4.2	Critical Analysis of the Research Item	8
2.1.4.3	Relationship to the Proposed Research Work	9
2.1.5	Comments in Source Code: A classification approach [2]	9
2.1.5.1	Summary of the Research Item	9
2.1.5.2	Critical Analysis of the Research Item	9
2.1.5.3	Relationship to the Proposed Research Work	10
2.1.6	Efficacy of Pretrained Architectures for Code Comment Useful- ness Prediction [11]	10

2.1.6.1	Summary of the Research Item	10
2.1.6.2	Critical Analysis of the Research Item	11
2.1.6.3	Relationship to the Proposed Research Work	11
2.1.7	Using Transformer-based Pre-trained Language Model for Automated Evaluation of Comments to Aid Software Maintenance [3] .	12
2.1.7.1	Summary of the Research Item	12
2.1.7.2	Critical Analysis of the Research Item	12
2.1.7.3	Relationship to the Proposed Research Work	13
2.1.8	Information Retrieval in Software Engineering utilizing a pre-trained BERT model [6]	13
2.1.8.1	Summary of the Research Item	13
2.1.8.2	Critical Analysis of the Research Item	13
2.1.8.3	Relationship to the Proposed Research Work	14
2.1.9	Exploring Transformer-Based Models for Automatic Useful Code Comments Detection [9]	14
2.1.9.1	Summary of the Research Item	14
2.1.9.2	Critical Analysis of the Research Item	14
2.1.9.3	Relationship to the Proposed Research Work	15
2.1.10	Identification of the Relevance of Comments in Codes Using Bag of Words and Transformer Based Models [12]	15
2.1.10.1	Summary of the Research Item	15
2.1.10.2	Critical Analysis of the Research Item	16
2.1.10.3	Relationship to the Proposed Research Work	16
2.1.11	Automatic comment usefulness judgement via SVM and ANN using contextual token representations [13]	17
2.1.11.1	Summary of the Research Item	17
2.1.11.2	Critical Analysis of the Research Item	17
2.1.11.3	Relationship to the Proposed Research Work	18
2.2	Analysis Summary of research items	18
3	Proposed Approach	23
3.1	Dataset Acquisition	24
3.2	Preprocessing	24
3.3	Feature Extraction	24
3.4	Oversampling for Low-Resource Languages	24
3.5	Model Selection and Evaluation	25
3.6	Training the Selected Model	25
3.7	Domain Post-Training	25
3.8	Evaluation Metrics	25
	References	28

List of Figures

3.1	Workflow Diagram	23
-----	----------------------------	----

List of Tables

1.1	Research Problem Statement	2
2.1	Summary of Research Items	19
2.2	Summary of Research Items (contd)	20
2.3	Summary of Research Items (contd)	21

Chapter 1

Introduction

Comments in the source code are an essential part of software engineering. They help developers understand the purpose and the functionality behind various code segments. These comments, normally written in natural languages, provide information that supports the maintenance of complex code. However, the presence of overabundant and unstructured code comments presents an obstacle—developers have to go through the extensive code documentation to find relevant information for understanding the functionality of the code. [10]

Hence, the classification of code comments comes out to be an important task, aiming to organize comments based on the type of information they convey. By grouping comments into categories such as functionality descriptions or metadata, developers can easily locate vital details for tasks such as debugging or feature enhancements. [4]. This classification process enhances overall code quality and offers the added benefit of improved documentation. By automating the process of classifying comments into different types, developers can optimize code comprehension, streamline maintenance efforts, and create a more efficient development process.

1.1 Problem Domain

The proposed research is to focus on the problem of inefficient code understanding within the software development process. Code comments are essential documentation for developers, aiding them in understanding the purpose and behavior of different code lines. However, the overabundant and unstructured comments make it difficult for developers to locate relevant information in time, which leads to delayed debugging, code reviews, and new feature development.

The core issue addressed by this research is to increase the accuracy than that of the baseline [4] for the accurate classification of code comments based on the information

of their semantic meaning. Without a system such as the proposed, developers have to manually search through extensive comments, resulting in misinterpretations of the comments which ultimately slows down the development. The proposed solution is to develop a model to classify code comments into categories, such as functionality descriptions, metadata, etc.

This solution aims to enhance code comprehension, enabling developers to quickly access the most relevant and important comments for their specific tasks. The beneficiaries of this research include software development teams, open-source communities, and organizations involved in software maintenance. With the implementation of this tool, stakeholders are set to benefit from improved productivity, efficient maintenance efforts, and a deeper understanding of issues in the code. Ultimately, this research aspires to optimize the coding process, allowing developers to work more effectively and maintain higher software quality.

1.2 Research Problem Statement

Problem	Description
The Problem of	The presence of overabundant, inaccurate, and unstructured source code comments.
Affects	Software Developers relying on the comments for comprehension, maintenance and debugging. Development processes requiring access to relevant documentation code completion, the identification of bugs, etc.
The result of which	Developers spend more time interpreting relevant comments, slowing down the maintenance and development process. Suboptimal or outdated code remains unaddressed which leads to the accumulation of technical debt.
Benefits of	Automatically classifying code comments into relevant categories would allow developers to quickly locate useful information. Developers can easily focus on replacing deprecated code and enhancing functionality without going through irrelevant documentation. Improved automation tools for code summarization and completion, which would enhance productivity across development workflows.

Table 1.1: Research Problem Statement

Chapter 2

Literature Review

2.1 Related Research

We conducted an SLR selecting 30 studies but we selected 11 relevant articles for our study. The 11 studies that we are going to focus on are listed below:

2.1.1: Performance Comparison of Binary Machine Learning Classifiers in Identifying Code Comment Types: An Exploratory Study [5]

2.1.2: Classifying Code Comments via Pre-trained Programming Language Model [8]

2.1.3: STACC: Code Comment Classification using SentenceTransformers [1]

2.1.4: Dopamin: Transformer-based Comment Classifiers through Domain Post-Training and Multi-level Layer Aggregation [4]

2.1.5: Comments in Source Code: A classification approach [2]

2.1.6: Efficacy of Pretrained Architectures for Code Comment Usefulness Prediction [11]

2.1.7: Using Transformer-based Pre-trained Language Model for Automated Evaluation of Comments to Aid Software Maintenance [3]

2.1.8: Information Retrieval in Software Engineering utilizing a pre-trained BERT model [6]

2.1.9: Exploring Transformer-Based Models for Automatic Useful Code Comments Detection [9]

2.1.10: Identification of the Relevance of Comments in Codes Using Bag of Words and Transformer Based Models [12]

2.1.11: Automatic comment usefulness judgement via SVM and ANN using contextual token representations [13]

2.1.1 Performance Comparison of Binary Machine Learning Classifiers in Identifying Code Comment Types: An Exploratory Study [5]

2.1.1.1 Summary of the Research Item

In this exploratory study, Indika et al. [5] examine the performance of various binary machine learning classifiers in identifying different types of code comments across Java, Python, and Pharo. Utilizing a dataset of 6,738 code comments, they developed 19 binary classifiers, each focused on specific comment categories. The study involved preprocessing the data through text normalization and oversampling to optimize classification results. Classifiers such as Decision Trees, Logistic Regression, Random Forest, and Support Vector Classifier were tested. The results indicate that the Linear SVC achieved the highest average F1 score of 0.5474, although the performance of the classifiers varied depending on the language and comment type.

2.1.1.2 Critical Analysis of the Research Item

Strengths of the Research Item:

- The study evaluates a broad range of machine learning classifiers, providing a comprehensive comparison of their performance.
- Utilization of the TF-IDF approach for converting text data into numerical features enhances the model's ability to process and classify comments effectively.
- The Linear SVC classifier achieved the highest average F1 score, indicating its strong performance in the context of code comment classification.

Weaknesses of the Research Item:

- Attention based deep learning models could have improved upon the performance of the tool leading to better precision and recall.
- Alternative preprocessing and oversampling techniques, features and hyperparameter tuning could lead to further improvement in the scores achieved.
- While the Linear SVC performed well overall, no single classifier excelled across all comment types.

2.1.1.3 Relationship to the Proposed Research Work

The exploratory study by Indika et al. [5] investigates the performance of various binary machine learning classifiers for identifying code comment types, revealing insights into the strengths and weaknesses of traditional classifiers. Our proposed research builds on this foundation by moving beyond traditional machine learning approaches to leverage advanced transformer models like CodeBERTa and GraphCodeBERT, which are specifically designed to capture the nuances of code and comment interactions across programming languages, including Java, Python, and Pharo. While Indika et al. highlight the efficacy of the Linear SVC, our work aims to improve upon these results by employing domain post-training and the EDISON oversampling technique to better handle class imbalance and enhance representation for low-resource languages. Additionally, our evaluation framework will provide a more comprehensive assessment of model performance across various categories, addressing the limitations identified in the study regarding inconsistent classifier performance. By integrating insights from Indika et al. with modern techniques, our research seeks to advance the field of code comment classification, improving accuracy and robustness in diverse programming contexts.

2.1.2 Classifying Code Comments via Pre-trained Programming Language Model [8]

2.1.2.1 Summary of the Research Item

In their study, Li et al. [8] propose a novel approach to classifying code comments using a pre-trained programming language model, CodeT5. This model is designed to enhance the classification performance of code comments by leveraging pre-existing knowledge from large software engineering datasets. The authors apply expert-predefined features during data preprocessing to improve accuracy, and they fine-tune CodeT5 on a dataset of comments from three programming languages: Java, Python, and Pharo. Their approach shows significant improvement over the baseline Random Forest model, achieving 0.72 precision, 0.60 recall, and a 0.65 F1-score—representing a notable increase in performance metrics. The study demonstrates the effectiveness of pre-trained models for code comment classification and highlights the potential for further advancements in this area through transfer learning.

2.1.2.2 Critical Analysis of the Research Item

Strengths of the Research Item:

- The study effectively utilizes the pre-trained CodeT5 model, leveraging domain-

specific knowledge to enhance classification accuracy.

- The inclusion of expert-predefined features proves fruitful as it allows the model to learn language patterns and semantics for better classification.
- Significant improvements in precision, recall, and F1-score demonstrate the superiority of the approach over traditional machine learning methods.

Weaknesses of the Research Item:

- The model's performance is limited in certain categories, such as Key Implementation Points and Class References, where results were weaker.
- Due to the model's somewhat reliance on expert predefined features, Python receives the worst F1 scores as it has a lesser amount of the aforementioned features.

2.1.2.3 Relationship to the Proposed Research Work

The study by Li et al. [8] introduces a novel approach to classifying code comments using the pre-trained CodeT5 model, achieving improved performance over traditional methods. Our proposed research similarly focuses on enhancing code comment classification through advanced transformer models like CodeBERTa and GraphCodeBERT, which are specifically designed for code and comment interactions. While Li et al. demonstrate the effectiveness of utilizing expert-predefined features, our approach aims to minimize reliance on such features by employing domain post-training and the EDISON oversampling technique to address dataset imbalance. This strategy is particularly beneficial for low-resource languages like Pharo and Python, where the availability of expert features may be limited. Furthermore, our research will incorporate a more extensive evaluation framework to assess model performance across various categories, thereby providing a comprehensive understanding of strengths and weaknesses. By building on the insights from Li et al. and addressing the identified limitations, our work aims to push the boundaries of automated code comment classification, ensuring robustness and accuracy across diverse programming languages.

2.1.3 STACC: Code Comment Classification using SentenceTransformers [1]

2.1.3.1 Summary of the Research Item

In their study, Al-Kaswan et al. [1] propose STACC (SentenceTransformer-Assisted Comment Classifiers), a binary classification system designed to categorize code comments. The research utilizes SetFit, a framework for few-shot fine-tuning of Sentence

Transformers, resulting in efficient and fast models. STACC was trained and tested on the NLBSE Code Comment Classification competition dataset, which includes three programming languages: Java, Python, and Pharo. The study's results demonstrate significant improvement over the baseline Random Forest classifiers, achieving an average F1 score of 0.74, which is an improvement of 139% compared to the baseline of 0.31.

STACC integrates Sentence-Transformers and is optimized through hyperparameter tuning using the Optuna backend, ultimately training classifiers for 19 categories. The system outperforms the baseline in nearly every category. The models and their training environment are publicly available, facilitating further experimentation and application.

2.1.3.2 Critical Analysis of the Research Item

Strengths of the Research Item:

- By leveraging Sentence Transformers through SetFit, the model achieves high accuracy with minimal data and training time.
- The model yields an improvement of 139% in F1 score compared to traditional models like Random Forest, demonstrating the effectiveness of transformer-based methods in code comment classification.

Weaknesses of the Research Item:

- Classifier performance is quite poor in some categories due to unbalanced dataset, which resulted in training which was not satisfactory
- Training time was increased when trained for 6 epochs, which did not increase performance much from when the model was trained for 2 epochs.
- Results may have been biased due to hyperparameter search which was done on only 1 category, as opposed to multiple categories.

2.1.3.3 Relationship to the Proposed Research Work

The study by Al-Kaswan et al. [1] introduces STACC, a binary classification system that leverages Sentence Transformers for code comment categorization, achieving significant improvements over traditional methods like Random Forest. Our proposed research aligns with this focus on advanced models, utilizing transformer architectures such as CodeBERTa and GraphCodeBERT to enhance code comment classification across multiple programming languages, including Java, Python, and Pharo. While STACC demonstrates impressive performance, it also highlights challenges related to dataset imbalance, which

can adversely affect classifier performance in certain categories. Our approach addresses this issue through the EDISON oversampling technique, ensuring better representation of underrepresented classes in the training dataset. Additionally, we focus on comprehensive evaluation metrics to provide deeper insights into model performance across various categories, overcoming some limitations of STACC's hyperparameter tuning process. By building on the successes of STACC and tackling its identified weaknesses, our research aims to further advance the field of automated code comment classification, improving robustness and accuracy in a multi-language context.

2.1.4 Dopamin: Transformer-based Comment Classifiers through Domain Post-Training and Multi-level Layer Aggregation [4]

2.1.4.1 Summary of the Research Item

In their study, Le Hai et al. [4] introduce Dopamin, a Transformer-based model designed to improve code comment classification through domain post-training and multi-level layer aggregation. By leveraging CodeBERT and transferring knowledge from high-resource languages like Java to low-resource ones like Python and Pharo, Dopamin achieves a notable F1-score of 0.74, surpassing the STACC baseline by 3%. This method enhances classification accuracy across various categories while maintaining efficient inference times.

The model's hierarchical layer aggregation method allows it to capture complex semantic information, leading to significant improvements in categories like Summary and Usage. Despite some challenges with certain categories, Dopamin demonstrates a robust and efficient approach to multi-language code comment classification.

2.1.4.2 Critical Analysis of the Research Item

Strengths of the Research Item:

- The authors proposed domain post-training which transfers knowledge across high-resource language (Java) to improve low-resource language (Python).
- The multi-level layer aggregation methodology helps Dopamin to understand the nuanced semantic information the comment carries.

Weaknesses of the Research Item:

- Dopamin showed weakness for 4 out of 7 categories of comments for Pharo possibly due to low data volume per category which means its approach is data hungry.

2.1.4.3 Relationship to the Proposed Research Work

The study by Le Hai et al. [4] introduces Dopamin, a transformer-based model that utilizes domain post-training and multi-level layer aggregation to enhance code comment classification, particularly in low-resource languages like Python and Pharo. Our proposed research aligns with this focus on improving classification across multiple languages, as we also leverage transformer models such as CodeBERTa and GraphCodeBERT. While Dopamin excels in transferring knowledge from high-resource languages, our approach further addresses the issue of data imbalance through the EDISON oversampling technique, which aims to enhance representation for underrepresented languages and comment categories. Additionally, we emphasize domain post-training to refine model performance specifically for code comments, similar to Dopamin’s methodology. However, our comprehensive evaluation framework aims to provide deeper insights into model performance across all comment categories, ensuring robustness even in low-data scenarios. By combining the strengths of Dopamin with our strategies for balancing datasets and optimizing model training, we seek to improve classification accuracy and generalization in automated code comment classification.

2.1.5 Comments in Source Code: A classification approach [2]

2.1.5.1 Summary of the Research Item

In their study, Shingala et al. [2] propose a supervised learning-based approach to classify code comments as either Useful or Not Useful, leveraging models such as Random Forest, SVM, and Logistic Regression. The authors use a dataset from FIRE 2022, containing 8657 comments, to train the models, with Random Forest achieving the best results at 0.83 precision and 0.84 recall. The study highlights the effectiveness of pre-processing techniques, such as stop word removal and word embeddings like Word2Vec and Bag-of-Words, to improve classification accuracy.

The paper also explores deep learning methods, such as Adam optimization, and suggests future improvements through transformer-based models and data augmentation to enhance the classification process and model performance.

2.1.5.2 Critical Analysis of the Research Item

Strengths of the Research Item:

- The study explores various supervised learning models, including traditional and deep learning approaches.

- Effective use of preprocessing techniques improves the representation of comments and contributes to feature extraction.
- Random Forest model performs well with an F1 score of 0.84.

Weaknesses of the Research Item:

- The paper could benefit from transformer-based models like BERT, which may enhance performance over current methods.
- Adversarial training could further improve model robustness.
- Data augmentation strategies could be explored to mitigate the limitations of the dataset.

2.1.5.3 Relationship to the Proposed Research Work

The study by Shingala et al. [2] presents a supervised learning approach for classifying code comments as Useful or Not Useful, achieving notable results with traditional models like Random Forest. Our proposed research builds on this foundation by focusing on transformer-based models such as CodeBERTa and GraphCodeBERT, which are designed to capture the complexities of code-comment interactions more effectively than the traditional methods used in their study. While Shingala et al. emphasize the importance of preprocessing techniques and traditional feature extraction methods, our approach not only incorporates these aspects but also addresses the limitations of dataset size and class imbalance through the EDISON oversampling technique. Moreover, while they suggest future work with transformer models, our research actively employs these advanced architectures to improve classification accuracy. By including domain post-training and a more comprehensive evaluation framework, we aim to enhance model robustness and performance across various programming languages and comment styles, thereby advancing the insights offered by Shingala et al. in the context of automated code comment classification.

2.1.6 Efficacy of Pretrained Architectures for Code Comment Usefulness Prediction [11]

2.1.6.1 Summary of the Research Item

In their study, Joshi et al. [11] evaluate the performance of various pretrained transformer architectures for predicting the usefulness of code comments. The authors aim to automate the identification of useful code comments, which helps improve code readability

and maintenance. They experimented with seven pretrained models, including BERT, ALBERT, CodeBERT, and XLNet, fine-tuning these models on a dataset of code comments labeled as ‘Useful’ or ‘Not Useful.’ The study revealed that certain models, such as XLNet and MPNet, performed well, achieving F1 scores of 85.66% and 83.88%, respectively. However, some models, such as CodeBERT and RoBERTa, exhibited anomalously high performance with nearly perfect scores, suggesting potential issues with the dataset or model overfitting. The study concludes that pretrained models show promise for code comment usefulness prediction, but further investigation is needed to address the anomalous results.

2.1.6.2 Critical Analysis of the Research Item

Strengths of the Research Item:

- CodeBERT and RoBERTa, which focus on understanding code semantics and natural language, highlight the potential for domain-specific pre-training.
- The evaluation approach, using multiple metrics like MCC and ROC-AUC, provides a comprehensive understanding of model performance.

Weaknesses of the Research Item:

- The abnormal performance of CodeBERT and RoBERTa raises concerns about overfitting.
- The lack of data augmentation methods limit the model’s ability to handle broader variations in comment styles and contexts.

2.1.6.3 Relationship to the Proposed Research Work

The study by Joshi et al. [11] evaluates various pretrained transformer models for predicting the usefulness of code comments, revealing promising results but also raising concerns about potential overfitting in models like CodeBERT and RoBERTa. Our proposed research builds on this foundation by focusing specifically on transformer-based models such as CodeBERTa and GraphCodeBERT, which are tailored for code-comment interactions. While Joshi et al. highlight the effectiveness of pretrained models, our approach incorporates domain post-training to refine model performance for code comment classification, thus addressing the limitations identified in their study regarding overfitting. Additionally, we implement the ADASYN oversampling technique to combat dataset imbalance, ensuring better representation of low-resource language comments, an area not thoroughly explored by Joshi et al. Our comprehensive evaluation metrics will also allow

for a more nuanced assessment of model performance across different comment types and contexts, further enhancing our understanding of how these models can be applied effectively in software maintenance tasks. Ultimately, our work seeks to advance the insights from Joshi et al. while mitigating the issues related to model performance and data variability.

2.1.7 Using Transformer-based Pre-trained Language Model for Automated Evaluation of Comments to Aid Software Maintenance [3]

2.1.7.1 Summary of the Research Item

In their study, Paul et al. [3] explore the use of transformer-based pre-trained language models for evaluating the usefulness of code comments to aid software maintenance. The authors experiment with models like BERT, CodeBERT, and GPT-2, fine-tuning them for the task of identifying whether a comment is useful or not in the context of the associated code. Their best-performing model, CodeBERT, achieves a high F1-score of 90.15% and accuracy of 91.21%. The study also incorporates AI techniques like attention visualization and LIME to provide insights into how these models classify comments. The results demonstrate the effectiveness of pre-trained language models in code comment usefulness prediction and suggest further applications in automated software maintenance systems.

2.1.7.2 Critical Analysis of the Research Item

Strengths of the Research Item:

- CodeBERT and RoBERTa, which focus on understanding code semantics and natural language, highlight the potential for domain-specific pre-training.
- The evaluation approach, using multiple metrics like MCC and ROC-AUC, provides a comprehensive understanding of model performance.

Weaknesses of the Research Item:

- The abnormal performance of CodeBERT and RoBERTa raises concerns about overfitting.
- The lack of data augmentation methods limit the model's ability to handle broader variations in comment styles and contexts.

2.1.7.3 Relationship to the Proposed Research Work

The study by Paul et al. [3] effectively demonstrates the potential of transformer-based models, particularly CodeBERT, for evaluating the usefulness of code comments, achieving high performance metrics. Our proposed research aligns with this approach by utilizing transformer models, such as CodeBERTa and GraphCodeBERT, to classify comments across diverse programming languages. While Paul et al. highlight the effectiveness of these models and incorporate explainable AI techniques for insight, our work goes a step further by addressing the challenge of dataset imbalance through the ADASYN oversampling technique, which is crucial for improving performance on low-resource languages. Additionally, our focus on domain post-training aims to refine model capabilities specifically for code comment classification, enhancing the understanding of domain-specific vocabulary. While Paul et al. raise concerns about overfitting, our comprehensive evaluation metrics will help monitor model performance more rigorously, ensuring robustness across varying comment styles and contexts. Overall, our approach seeks to build on the insights from Paul et al. while addressing limitations and expanding the application of transformer models in software maintenance tasks.

2.1.8 Information Retrieval in Software Engineering utilizing a pre-trained BERT model [6]

2.1.8.1 Summary of the Research Item

In their study, Ghosh et al. [6] address the challenge of classifying code comments as useful or not useful in the context of software engineering. The authors utilize the IRSE (Information Retrieval in Software Engineering) dataset from the FIRE 2022 shared task, applying the RoBERTa model, a pre-trained variant of BERT, to classify code comments based on their usefulness. They fine-tune RoBERTa on 8047 comment and code pairs and achieve an F1 macro score of 0.9047. The model is trained with hyperparameters such as a learning rate of $1e-5$, batch size of 5, and maximum sequence length of 512 tokens. Their approach demonstrates a strong performance in automatically evaluating code comments, securing the second position in the competition.

2.1.8.2 Critical Analysis of the Research Item

Strengths of the Research Item:

- The use of the pre-trained RoBERTa model, which is optimized for text classification tasks, leads to a high F1 macro score of 0.9047.

- The model effectively handles the classification task without requiring extensive feature engineering, showcasing the power of transformer-based architectures.
- The authors' detailed hyperparameter tuning and experimentation strengthen the model's performance in this context.

Weaknesses of the Research Item:

- The study is limited by the model's inability to process longer sequences effectively due to the 512-token limitation, which could impact performance on more complex code-comment pairs.

2.1.8.3 Relationship to the Proposed Research Work

The connection between Research Item 1 and the present study is explored, highlighting how the findings, methodologies, or gaps identified in the literature inform and shape the current research approach and objectives.

2.1.9 Exploring Transformer-Based Models for Automatic Useful Code Comments Detection [9]

2.1.9.1 Summary of the Research Item

In their study, Das et al. [9] explore transformer-based models for detecting Useful code comments, introducing a fusion model that combines BERT and CodeBERT. Their model, BERT+CodeBERT(PP), leverages both code comments and associated code snippets to improve classification accuracy, achieving a Macro F1-score of 90.739 in the IRSE shared task at FIRE 2022, outperforming previous models. The study emphasizes the importance of incorporating code snippets for more accurate comment classification and utilizes post-processing techniques, such as relabeling short comments as Not-Useful, to further enhance model performance.

By leveraging BERT for natural language features and CodeBERT for programming-specific features, the fusion model successfully outperforms standalone models, demonstrating the potential of combining natural language processing with programming language models for code comment classification.

2.1.9.2 Critical Analysis of the Research Item

Strengths of the Research Item:

- The approach effectively combines BERT and CodeBERT to leverage both comment text and associated code snippets.
- The post-processing step for relabeling short comments improves model performance.
- The fusion-based model outperforms traditional models, achieving high F1-scores across the dataset.

Weaknesses of the Research Item:

- The reliance on post-processing for short comments highlights a limitation in detecting Useful comments based solely on model outputs.

2.1.9.3 Relationship to the Proposed Research Work

The study by Das et al. [9] demonstrates the efficacy of a fusion model combining BERT and CodeBERT for detecting Useful code comments, achieving impressive performance by integrating code snippets with comment text. In our proposed research, we similarly focus on transformer-based models, specifically CodeBERTa and GraphCodeBERT, to enhance comment classification accuracy across multiple programming languages. While Das et al. highlight the importance of incorporating associated code snippets, our approach aims to further leverage the strengths of dedicated models for both code and comment interactions without heavily relying on post-processing techniques. By addressing dataset imbalance through the ADASYN oversampling technique and implementing domain post-training, we seek to improve model performance in a more holistic manner. Additionally, our comprehensive evaluation metrics will provide deeper insights into model effectiveness across different comment categories, building upon the foundational insights from Das et al. while striving to overcome their reliance on post-processing for optimal results.

2.1.10 Identification of the Relevance of Comments in Codes Using Bag of Words and Transformer Based Models [12]

2.1.10.1 Summary of the Research Item

In their study, Sruthi et al. [12] explore the use of Bag of Words and transformer-based models for classifying the relevance of code comments in software. The authors apply traditional machine learning models like Random Forest, SVM, and Logistic Regression using TF-IDF and Entropy-based features, alongside transformer models such as BERT,

RoBERTa, and ALBERT, fine-tuning them on a training dataset from FIRE 2022. Surprisingly, their results show that Bag of Words models outperform transformer-based models, with SVM using entropy-based features achieving the best performance on the test dataset.

The authors conclude that the relatively small size of the training corpus is a limitation for transformer models and propose the need for novel embeddings specifically tailored to software development tasks to improve future performance.

2.1.10.2 Critical Analysis of the Research Item

Strengths of the Research Item:

- Successfully implemented a variety of models, providing a comprehensive comparison of Bag of Words and transformer-based models.
- The use of TF-IDF and Entropy-based features offers valuable insights into traditional feature extraction methods.
- The study highlights the limitations of current transformer models when applied to small, domain-specific datasets like software comments.

Weaknesses of the Research Item:

- The transformer models, including BERT and RoBERTa, underperform due to the limited size of the training corpus.
- The reliance on general-purpose pre-trained models, such as BERT, may not capture the specific semantics of code comments.

2.1.10.3 Relationship to the Proposed Research Work

The study by Sruthi et al. [12] investigates the classification of code comments using both Bag of Words and transformer-based models, revealing that traditional methods can outperform transformers in certain contexts due to dataset limitations. In contrast, our proposed approach prioritizes advanced transformer models like CodeBERTa and Graph-CodeBERT, which are specifically designed for code and comment interactions, aiming to leverage their deeper semantic understanding. While Sruthi et al. highlight the limitations of using general-purpose models on small, domain-specific datasets, our strategy includes domain post-training to refine model performance on code comments. Furthermore, we address the challenge of dataset imbalance through the ADASYN oversampling technique for low-resource languages, an aspect not covered in their study. By employing

a more comprehensive set of evaluation metrics and focusing on domain-specific vocabulary, our approach seeks to improve classification accuracy across diverse programming languages, building on the insights provided by Sruthi et al. while aiming to overcome their identified limitations.

2.1.11 Automatic comment usefulness judgement via SVM and ANN using contextual token representations [13]

2.1.11.1 Summary of the Research Item

In their study, Sahu et al. [13] propose a machine learning-based framework for classifying code comments as Useful or Not Useful by leveraging Support Vector Machine (SVM) and Artificial Neural Networks (ANN) with ELMO-based contextual token representations. The system is trained on a dataset from the IRSE shared task at FIRE 2022, where it achieves an F1-score of 0.88 using an SVM model with an RBF kernel. The authors generate separate representations for both code comments and their associated code snippets using ELMO embeddings, later concatenating them for model training.

The study demonstrates the effectiveness of combining traditional machine learning algorithms with modern neural representations for classifying code comments, yielding high accuracy and precision in predicting the relevance of comments.

2.1.11.2 Critical Analysis of the Research Item

Strengths of the Research Item:

- Sahu et al. effectively utilize ELMO-based contextual representations to capture the semantics of comments and code.
- The SVM with RBF kernel outperforms other models, achieving high accuracy and F1-scores.
- The study offers a comprehensive comparison of SVM and ANN, showcasing the robustness of traditional machine learning approaches when enhanced with modern embeddings.

Weaknesses of the Research Item:

- The reliance on ELMO embeddings, while effective, may not capture the deeper semantics of code that transformer models could provide.
- ANN underperforms compared to SVM, suggesting limited effectiveness for the neural model in this task.

2.1.11.3 Relationship to the Proposed Research Work

The research by the authors focuses on classifying code comments using ELMO embeddings with traditional machine learning models like SVM and ANN, achieving notable accuracy. In contrast, our proposed approach leverages advanced transformer-based models such as CodeBERTa and GraphCodeBERT, which capture deeper semantic relationships in code and comments. We also address dataset imbalance through the ADASYN oversampling technique, particularly for low-resource languages, a consideration absent in Sahu et al.'s work. Additionally, our evaluation incorporates a broader range of metrics, including accuracy and F1-score, to ensure a comprehensive assessment of performance. Finally, we emphasize domain post-training to refine the model's understanding of programming-specific vocabulary, further enhancing its classification capabilities across diverse programming languages.

2.2 Analysis Summary of research items

Tabular representation with discussion of the critical analysis of the research items discussed in the previous section of this chapter.

	Dataset Size & Source	Preprocessing Techniques	Classifiers Trained	Research Objective	Limitations	Performance Measures
[5]	6738 class comment sentences NLBSE 2023	Text normalization, stemming, removal of stopwords	Naive Bayes, SVM, Random Forest, Decision Trees, KNN, MLP, Logistic Regression	Compare the performance of binary classifiers on comment classification	Limited to binary classifiers, no exploration of deep learning models	Precision, Recall, F1-measure
[8]	6738 class comment sentences NLBSE 2023	Tokenization, Expert-predefined feature extraction, Tagging features	CodeT5 (19 Binary Classifiers)	Classify code comments into multiple categories across 3 languages	Struggles with low-resource categories (Pharo, Python); some categories harder to classify accurately	Precision, Recall, F1-measure
[1]	6738 class comment sentences NLBSE 2023	Tokenization, Few-shot learning with SetFit	all-mpnet-base-v2 (19 Binary Classifiers)	Classify code comments using Sentence Transformers	Dataset imbalances, limited to three programming languages	Precision, Recall, F1-measure
[4]	82089 class comment sentences NLBSE 2024	Concatenating class name and comment sentence	CodeBERT (19 Binary Classifiers)	Improve comment classification using domain post-training and multi-level layer aggregation	Limited performance on Pharo language due to lower data volume	Precision, Recall, F1-measure

Table 2.1: Summary of Research Items

[2]	8657 records for training and 1000 records for testing	Stop word removal, Text normalization, Removal of URLs	Support Vector Machines (SVM), Naive Bayes, Random Forest, and Decision Trees.	Classify code comments in C and C++ as "Useful" or "Not Useful"	Limited language coverage, focus on C and C++ comments	Precision, Recall, F1-Score
[11]	5,354 training samples, 678 test samples	Stopword removal, text normalization, word embeddings	XLNet, MPNet, CodeBERT, GPT-2	Classify code comments in C and C++ as "Useful" or "Not Useful"	Limited language coverage, focus on C and C++ comments	Precision, Recall, F1-Score, Accuracy
[3]	Data from FIRE 2022	Tokenization, camel-case splitting, symbol removal	BERT, CodeBERT (RoBERTa), GPT-2	Automate the classification of useful code comments using pre-trained models	Focused on a specific dataset (FIRE 2022), limited explainability of transformer models	Precision, Recall, F1-Score, Accuracy
[6]	Data from FIRE 2022	Removing punctuation, tokenization	BERT, RoBERTa, ALBERT	To evaluate transformer-based models for code comment relevance classification	Limited performance on small datasets, pre-trained models are not specialized for code snippets	Precision, Recall, F1-Score, Accuracy

Table 2.2: Summary of Research Items (contd)

[9]	Data from FIRE 2022	Tokenization, special character removal, concatenation of code and comment embeddings	BERT, CodeBERT, BERT+CodeBERT	To detect useful comments by leveraging transformer-based models for combined code and comment embeddings	Dataset too small for transformer models, performance drops in complex scenarios	Precision, Recall, F1-Score, Accuracy
[12]	Data from FIRE 2022	Tokenization, feature extraction with TF-IDF, Entropy-based features	Random Forest, SVM, Logistic Regression, BERT, RoBERTa, ALBERT	To compare the effectiveness of Bag of Words vs transformer-based models for comment relevance classification	Transformer models underperform due to small dataset size	Precision, Recall, F1-Score, Accuracy
[13]	Data from FIRE 2022	Tokenization, punctuation removal, ELMO-based word embeddings for code and comments	SVM (Linear, RBF Kernel), ANN	To classify comment usefulness using machine learning models trained on contextual word embeddings	ELMO embeddings might not capture deeper semantics of code, ANN underperforms compared to SV	Precision, Recall, F1-Score, Accuracy

Table 2.3: Summary of Research Items (contd)

Chapter 3

Proposed Approach

In this approach, we enhance comment classification using transformer models and optimized preprocessing to improve performance across various programming languages.

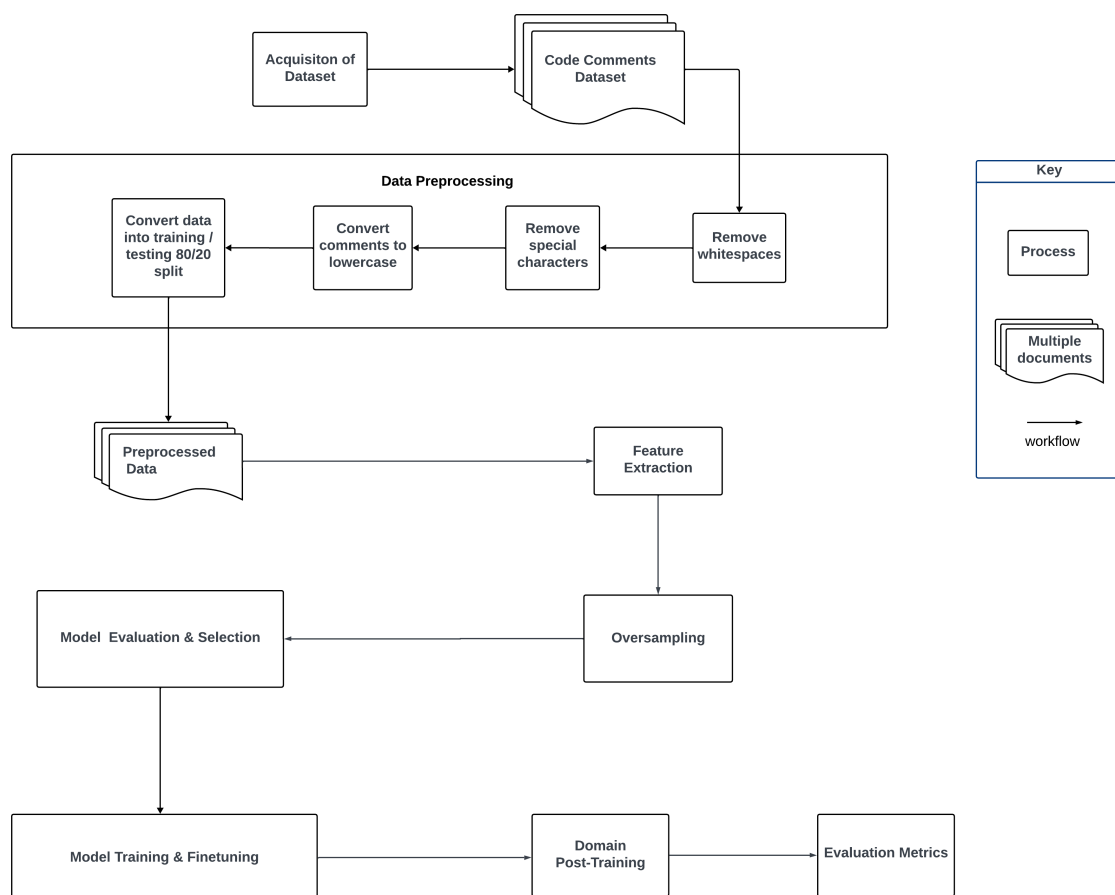


Figure 3.1: Workflow Diagram

3.1 Dataset Acquisition

We will acquire the dataset from the **NLBSE Tool Competition Website**, which provides a large corpus of code comments across multiple programming languages such as Java, Python, and Pharo. This dataset will be used for training, validation, and testing of our models for comment classification tasks.

3.2 Preprocessing

The preprocessing steps involve cleaning the raw dataset to ensure consistency and better model performance. Key steps include:

- **Removing Null Spaces:** Identifying and removing records with missing data to maintain data integrity.
- **Removing Special Characters:** Filtering out unnecessary symbols and characters that do not contribute to semantic understanding.
- **Converting to Lowercase:** Standardizing the text by converting all characters to lowercase, making the data uniform.
- **Data Splitting:** Splitting the cleaned dataset into **training** and **testing** sets, ensuring a representative sample is used for both.

3.3 Feature Extraction

After the preprocessing is done, we will extract features by converting code comments into vectorized representations. We will be leveraging techniques such as TFIDF for this task.

3.4 Oversampling for Low-Resource Languages

To address the imbalance in datasets, especially for low-resource languages like Pharo, we will apply the **EDISON** oversampling technique. This approach will help balance the dataset, ensuring that underrepresented classes receive sufficient attention during training, thus improving any selected model's ability to generalize across multiple languages.

3.5 Model Selection and Evaluation

We will experiment with multiple transformer-based models such as:

- **CodeBERTa**: Refined pre-training focused on code-comment interactions.
- **GraphCodeBERT**: Incorporates code structure through graph representations.
- **PLBART**: Excels at code generation and summarization tasks.

Each model will be evaluated based on its performance on the validation dataset.

3.6 Training the Selected Model

Once the best model has been selected and features have been extracted, we will proceed with the model training. The training process will focus on optimizing the model's performance by tuning key parameters such as batch size, learning rate and number of epochs.

To prevent overfitting, we will monitor performance using early stopping techniques, ensuring the model generalizes well across unseen data. **Hyperparameter optimization** will be applied to fine-tune the model for better accuracy across multiple programming languages. Additionally, **multi-level layer aggregation** will be employed to improve the model's ability to capture complex patterns in the comment text.

The training will utilize the augmented dataset, ensuring that both high-resource and low-resource language comments are well-represented.

3.7 Domain Post-Training

We will conduct **domain post-training** for the selected model, refining it to specialize in code comment classification rather than general code and natural language data. This step will help the model better understand domain-specific vocabulary, especially in the context of programming languages and comments, further improving its performance on classification tasks.

3.8 Evaluation Metrics

The following metrics will be used to evaluate model performance:

- **Accuracy:** The overall correctness of predictions.
- **F1-score:** The harmonic mean of precision and recall, particularly important for handling imbalanced datasets.
- **Precision and Recall:** To understand how well the model classifies positive comment categories.

By using these metrics, we will ensure that the selected model performs well across all comment categories and programming languages.

Bibliography

- [1] Ali Al-Kaswan, Maliheh Izadi, and Arie Van Deursen. Stacc: Code comment classification using sentencetransformers. In *2023 IEEE/ACM 2nd International Workshop on Natural Language-Based Software Engineering (NLBSE)*, pages 28–31, 2023.
- [2] Namrata Shroff Amisha Shingala. Comments in source code: A classification approach. In Kripabandhu Ghosh and Mitra [7], pages 74–79.
- [3] Rudrani Paul Debjyoti Paul, Bitan Biswas. Using transformer-based pre-trained language model for automated evaluation of comments to aid software maintenance. In Kripabandhu Ghosh and Mitra [7], pages 39–52.
- [4] Nam Le Hai and Nghi D. Q. Bui. Dopamin: Transformer-based comment classifiers through domain post-training and multi-level layer aggregation. In *2024 IEEE/ACM International Workshop on Natural Language-Based Software Engineering (NLBSE)*, pages 61–64, 2024.
- [5] Amila Indika, Peter Y. Washington, and Anthony Peruma. Performance comparison of binary machine learning classifiers in identifying code comment types: An exploratory study. In *2023 IEEE/ACM 2nd International Workshop on Natural Language-Based Software Engineering (NLBSE)*, pages 20–23, 2023.
- [6] Apurbalal Senapati Koyel Ghosh. Information retrieval in software engineering utilizing a pre-trained bert model. In Kripabandhu Ghosh and Mitra [7], pages 24–28.
- [7] Prasenjit Majumder Kripabandhu Ghosh, Thomas Mandl and Mandar Mitra, editors. *Working Notes of FIRE 2022 - Forum for Information Retrieval Evaluation*, number 994 in CEUR Workshop Proceedings, Aachen, 2022.
- [8] Ying Li, Haibo Wang, Huaien Zhang, and Shin Hwei Tan. Classifying code comments via pre-trained programming language model. In *2023 IEEE/ACM 2nd International Workshop on Natural Language-Based Software Engineering (NLBSE)*, pages 24–27, 2023.

- [9] Subhadeep Chatterjee Mithun Das. Exploring transformer-based models for automatic useful code comments detection. In Kripabandhu Ghosh and Mitra [7], pages 15–23.
- [10] Pooja Rani, Arianna Blasi, Nataliia Stulova, Sebastiano Panichella, Alessandra Gorla, and Oscar Nierstrasz. A decade of code comment quality assessment: A systematic literature review. *Journal of Systems and Software*, 195:111515, 2023.
- [11] Aditya Hari Abhijeeth Singam Vasudeva Varma Sagar Joshi, Sumanth Balaji. Efficacy of pretrained architectures for code comment usefulness prediction. In Kripabandhu Ghosh and Mitra [7], pages 29–33.
- [12] Tanmay Basu Sruthi S. Identification of the relevance of comments in codes using bag of words and transformer based models. In Kripabandhu Ghosh and Mitra [7], pages 60–65.
- [13] Ayan Das Yogesh Kumar Sahu. Automatic comment usefulness judgement via svm and ann using contextual token representations. In Kripabandhu Ghosh and Mitra [7], pages 66–73.